

Pandas Cheat Sheet

pandas con las operaciones más comunes y útiles:

Importar pandas

```
import pandas as pd
```

Crear DataFrames

```
# Desde un diccionario
df = pd.DataFrame({'A': [1, 2, 3], 'B': ['a', 'b', 'c']})

# Desde una lista de listas
df = pd.DataFrame([[1, 'a'], [2, 'b'], [3, 'c']],
                  columns=['A', 'B'])

# Desde un archivo CSV
df = pd.read_csv('archivo.csv')

# Desde un archivo Excel
df = pd.read_excel('archivo.xlsx')
```

Inspeccionar datos

```
df.head()    # Primeras 5 filas
df.tail()    # Últimas 5 filas
df.info()    # Información general del DataFrame
df.describe() # Estadísticas descriptivas
df.shape     # Dimensiones (filas, columnas)
df.columns   # Nombres de las columnas
df.dtypes    # Tipos de datos de las columnas
```

Seleccionar datos

```
df['columna'] # Seleccionar una columna
df[['col1', 'col2']] # Seleccionar múltiples columnas
```

```
df.loc[0] # Seleccionar fila por etiqueta
df.iloc[0] # Seleccionar fila por posición
df.loc[0, 'columna'] # Seleccionar valor específico
df[df['columna'] > 5] # Filtrar filas
```

Manipular datos

```
df['nueva_columna'] = df['columna'] * 2 # Crear nueva
columna
df.drop('columna', axis=1, inplace=True) # Eliminar
columna
df.rename(columns={'viejo_nombre': 'nuevo_nombre'},
inplace=True) # Renombrar columna
df.sort_values('columna', ascending=False) # Ordenar
df.reset_index(drop=True, inplace=True) # Resetear índice
```

Manejar valores nulos

```
df.isnull().sum() # Contar valores nulos
df.dropna() # Eliminar filas con valores nulos
df.fillna(0) # Rellenar valores nulos con 0
```

Agrupar y agregar datos

```
df.groupby('columna').mean() # Agrupar y calcular media
df.groupby(['col1', 'col2']).sum() # Agrupar por múltiples
columnas
df.pivot_table(values='valor', index='indice',
columns='columnas') # Tabla pivote
```

Combinar DataFrames

```
pd.concat([df1, df2]) # Concatenar verticalmente
pd.merge(df1, df2, on='columna_clave') # Unir DataFrames
```

Operaciones de tiempo

```
pd.to_datetime(df['fecha']) # Convertir a datetime
df.resample('M').mean() # Remuestreo por mes (requiere
índice de tiempo)
```

Estadísticas y cálculos

```
df['columna'].mean()    # Media
df['columna'].median()  # Mediana
df['columna'].mode()    # Moda
df['columna'].std()     # Desviación estándar
df['columna'].corr(df['otra_columna']) # Correlación
```

Exportar datos

```
df.to_csv('archivo.csv', index=False) # Exportar a CSV
df.to_excel('archivo.xlsx', index=False) # Exportar a Excel
```

Visualización rápida

```
df.plot(kind='bar')    # Gráfico de barras
df.hist()              # Histograma
df.boxplot()           # Diagrama de caja
```

Esta cheat sheet cubre las operaciones más comunes en pandas. Recuerda que pandas es una librería muy extensa y tiene muchas más funcionalidades. Para casos específicos o más avanzados, siempre es bueno consultar la documentación oficial de pandas.

NumPy Cheat Sheet

Importar NumPy

```
import numpy as np
```

Crear arrays

```
a = np.array([1, 2, 3]) # 1D array
b = np.array([[1, 2, 3], [4, 5, 6]]) # 2D array
np.zeros((3, 3)) # Array 3x3 de ceros
np.ones((2, 2)) # Array 2x2 de unos
np.eye(3) # Matriz identidad 3x3
np.arange(10) # Array de 0 a 9
np.linspace(0, 1, 5) # 5 números equidistantes entre 0 y 1
np.random.rand(3, 3) # Array 3x3 de números aleatorios
entre 0 y 1
```

Propiedades de arrays

```
a.shape # Dimensiones del array
a.dtype # Tipo de datos
a.size # Número total de elementos
a.ndim # Número de dimensiones
```

Indexación y slicing

```
a[0] # Primer elemento
a[-1] # Último elemento
a[0:2] # Primeros dos elementos
b[0, 1] # Elemento en la primera fila, segunda columna
b[:, 1] # Segunda columna completa
```

Operaciones básicas

```
a + b # Suma elemento a elemento
a - b # Resta elemento a elemento
a * b # Multiplicación elemento a elemento
a / b # División elemento a elemento
a ** 2 # Elevar al cuadrado cada elemento
```

```
np.dot(a, b) # Producto punto
```

Funciones matemáticas

```
np.sum(a) # Suma de todos los elementos  
np.mean(a) # Media  
np.median(a) # Mediana  
np.std(a) # Desviación estándar  
np.max(a) # Valor máximo  
np.min(a) # Valor mínimo  
np.argmax(a) # Índice del valor máximo  
np.argmin(a) # Índice del valor mínimo
```

Funciones trigonométricas

```
np.sin(a)  
np.cos(a)  
np.tan(a)
```

Álgebra lineal

```
np.linalg.inv(a) # Inversa de una matriz  
np.linalg.det(a) # Determinante  
np.linalg.eig(a) # Autovalores y autovectores
```

Manipulación de formas

```
a.reshape(3, 3) # Cambiar forma a 3x3  
a.flatten() # Aplanar a 1D  
np.transpose(a) # Transponer  
np.concatenate((a, b)) # Concatenar arrays
```

Operaciones lógicas

```
a > 5 # Comparación elemento a elemento  
np.all(a > 5) # ¿Todos los elementos son mayores que 5?  
np.any(a > 5) # ¿Algún elemento es mayor que 5?  
np.where(a > 5) # Índices donde los elementos son mayores  
que 5
```

Estadísticas

```
np.corrcoef(a, b) # Coeficiente de correlación  
np.cov(a, b)     # Matriz de covarianza
```

Operaciones de conjuntos

```
np.unique(a)      # Elementos únicos  
np.intersect1d(a, b) # Intersección  
np.union1d(a, b)  # Unión
```

Guardar y cargar

```
np.save('archivo', a) # Guardar array  
np.load('archivo.npy') # Cargar array
```

Operaciones de broadcasting

```
a + 5 # Suma 5 a cada elemento  
a * np.array([1, 2, 3]) # Multiplica elemento a elemento
```

Esta cheat sheet cubre las operaciones más comunes en NumPy. NumPy es una librería muy potente con muchas más funcionalidades, especialmente para cálculos numéricos y operaciones con matrices. Para casos específicos o más avanzados, es recomendable consultar la documentación oficial de NumPy.

Scikit-learn Cheat Sheet

```
from sklearn import datasets, metrics, model_selection, preprocessing
```

```
# Cargar datos
iris = datasets.load_iris()
X, y = iris.data, iris.target

# Dividir datos
X_train, X_test, y_train, y_test =
model_selection.train_test_split(X, y, test_size=0.3)

# Preprocesamiento
scaler = preprocessing.StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Modelos
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

model = LogisticRegression()
model.fit(X_train_scaled, y_train)

# Predicción y evaluación
y_pred = model.predict(X_test)
print(metrics.accuracy_score(y_test, y_pred))
print(metrics.classification_report(y_test, y_pred))
```

TensorFlow/Keras Cheat Sheet

```
import tensorflow as tf
from tensorflow import keras

# Crear modelo secuencial
model = keras.Sequential([
    keras.layers.Dense(64, activation='relu',
input_shape=(784,)),
    keras.layers.Dense(10, activation='softmax')
])

# Compilar modelo
model.compile(optimizer='adam',
loss='categorical_crossentropy', metrics=['accuracy'])

# Entrenar modelo
model.fit(X_train, y_train, epochs=10, batch_size=32,
validation_split=0.2)

# Evaluar modelo
test_loss, test_acc = model.evaluate(X_test, y_test)

# Predicción
predictions = model.predict(X_test)
```

PyTorch Cheat Sheet

```
python
import torch
import torch.nn as nn
import torch.optim as optim

# Definir modelo
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(784, 64)
        self.fc2 = nn.Linear(64, 10)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
```

```

        x = self.fc2(x)
        return x

model = Net()

# Definir pérdida y optimizador
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

# Entrenar modelo
for epoch in range(10):
    for data, target in train_loader:
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()

# Evaluar modelo
model.eval()
with torch.no_grad():
    correct = 0
    for data, target in test_loader:
        output = model(data)
        pred = output.argmax(dim=1, keepdim=True)
        correct +=
pred.eq(target.view_as(pred)).sum().item()

```

Matplotlib Cheat Sheet

```
import matplotlib.pyplot as plt
```

```
# Gráfico de líneas
```

```
plt.plot(x, y)
plt.title('Título')
plt.xlabel('Eje X')
plt.ylabel('Eje Y')
plt.show()
```

```
# Gráfico de dispersión
```

```
plt.scatter(x, y)
plt.show()
```

```
# Histograma
```

```
plt.hist(data, bins=20)
plt.show()
```

```
# Subplots
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 5))
ax1.plot(x, y1)
ax2.plot(x, y2)
plt.show()
```

Seaborn Cheat Sheet

```
import seaborn as sns

# Gráfico de dispersión
sns.scatterplot(x='columna1', y='columna2', data=df)

# Gráfico de líneas
sns.lineplot(x='x', y='y', data=df)

# Histograma
sns.histplot(data=df['columna'])

# Heatmap
sns.heatmap(data=df.corr(), annot=True)

# Pair plot
sns.pairplot(df)

# Box plot
sns.boxplot(x='categoria', y='valor', data=df)
```

Estas cheat sheets cubren las operaciones básicas más comunes en cada una de estas bibliotecas. Recuerda que cada una de estas bibliotecas es muy extensa y tiene muchas más funcionalidades. Para casos específicos o más avanzados, siempre es recomendable consultar la documentación oficial de cada biblioteca.